Node.Js

Del siguiente enlace podemos obtener node de diferentes maneras, la más recomendada es mediante el .msi: https://nodejs.org/es/download
Una vez instalado obtenemos por defecto npm (el gestor de dependencia (packages) por defecto de node), pero tiene unas cuantas desventajas, siendo estas no tener buena gestión de archivos en cuanto a seguridad y lo mas problematico, el espacio en disco al instalar dependencias en múltiples proyectos, la solución es instalar otro gestor de paquetes, tenemos yarn o pnpm. El mejor, que nos ofrece una capa extra de seguridad con su nueva versión y tiene un sistema de gestión de paquetes mediante enlaces simbólicos, pnpm es una buena opción.
Para instalar pnpm después de instalar node.
Abrir el cmd o powershell como administrador y ejecutar el siguiente comando.
corepack enable pnpm
Luego ejecutar ***pnpm —version***
Luego dar a confirmar al mensaje que aparece sobre permitir el uso de pnpm con Y.
Para tener la versión más actual de pnpm (la capa extra de seguridad está en la versión 11.0.0 para adelante) ejecutar el siguiente comando con la powershell o cmd en administrador.
corepack prepare pnpm@latest —activate
Una vez instalado ya podemos usar pnpm para ejecutar proyecto con js.

- ## Comenzar un proyecto

Para comenzar un proyecto nos situamos en una carpeta donde tendremos el código.
pnpm init → para crear un archivo package.json donde se tendrá las configuraciones para ejecutar el proyecto.

package.json

```
{
  "name": "proyecto",  //nombre del archivo
  "version": "1.0.0", //versión del proyecto
  "description": "", //descripción del proyecto
  "main": "index.js", //archivo principal al ejecutar el proyecto
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "packageManager": "pnpm@11.1.2" //version de pnpm en el proyecto
}
```

Para más comodidad poner en script el siguiente valor:

```
"scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
}
```

Luego crear un archivo index.js donde comenzará la ejecución del código al ejecutar el siguiente comando.

node index.js

// si ponemos start en script de package.json ya no usamos ***node index.js*** sino ***pnpm start*** esto nos ayuda a ahorrar el comando en la terminal si es que la ruta es más larga.

Si no instalamos dependencias se ejecutará javascript como backend (desde la consola).

➔ Para instalar dependencias: ***pnpm add #nombre del paquete***.

➔ Para quitar dependencias: ***pnpm remove #nombre del paquete***.

➔ Para limpiar por completo las dependencias de un proyecto. ejecutar

1. ***pnpm remove #nombre*** → a cada dependencia agregada anteriormente.
2. ***pnpm prune*** → para tener disposición de los archivos residuales de las dependencias.
3. ***pnpm store prune*** → para limpiar los archivos residuales.

Al instalar dependencias se agregara una nueva carpeta en nuestra carpeta principal, llamada node_modules, aquí se guardaran las dependencias en el caso de npm, se instalarán de 0 (más espacio en disco), con pnpm solo tendremos un enlace simbólico (similar a un acceso directo).

pnpm install → comando para instalar dependencias, similar a ***pnpm add***, la diferencia es que ***pnpm add*** instala las dependencias en un proyecto nuevo, en cambio ***pnpm install*** es para instalar las dependencias que ya estaban en un package.json y en las versiones especificadas, con add casi siempre es la actual.

Si tuviéramos esto en un package.json, que nos han compartido

```
packageManager": "pnpm@11.1.2",
"dependencies": {
  express": "^5.2.1",
  "mysql2": "^3.22.3",
}
```

De normal tendríamos que instalar cada dependencia a mano para usarlo, pero con ***pnpm install*** se instalan todas las depedencias que estén indicadas en el atributo dependencies del package.json del proyecto en las mismas versiones.

//Tu versión de pnpm debe ser la misma que la que indicar packageManager, sino te saldrá la opción de instalarla para poder usarla, para evitarnos problemas mejor cambia la versión de pnpm del package.json, a mano, a la tuya (sino te acuerdas ejecuta ***pnpm –version*** para verla) y luego ejecuta ***pnpm install*** y a probar codigo.

- **Para hacer API Rest con Js.**

Creamos una carpeta donde tendremos una carpeta.
Ejecutamos ***pnpm init*** para crear el package.json para las configuraciones.
Agregamos las dependencias requeridas con:
pnpm add express mysql2
Express para el servidor y mysql2 para conectar la base de datos con js.
➔ Ver las dependencias:
En el package.json aparecerán en dependencias en el atributo dependencies.

```
{
  "name": "northwind",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "packageManager": "pnpm@11.1.2",
  "dependencies": {
    "express": "^5.2.1",
    "mysql2": "^3.22.3",
  }
}
```

También podemos ejecutar ***pnpm list***.
Luego creamos un archivo index.js donde tendremos toda la configuración principal.
Importamos los módulos

```
const express = require('express'); //exportamos express con
require.
const app = express(); //creamos app
const path = require('path'); //para mejor manejo de las rutas
```

Creamos app para la gestión del servidor.
Luego creamos los middleware, que son configuraciones para nuestro servidor.

```
app.use(express.json()); //permite devolver json en las API.
app.use(express.urlencoded({extended: false})); //ayuda con el
paso de parámetros en el método post.
```

➔ Configurar la base de datos.

Ya sea en un archivo aparte del index.js o en el mismo, en este caso usaremos otro llamado db.js.

```
const mysql = require('mysql2/promise') //importamos mysql2 con promise para mejor uso de las querys con el uso de promesas.
const conn = mysql.createPool({//método que ayuda al uso de las querys.
    host: 'localhost',
    user: 'root',
    password: '',
    database: 'productos_db' //ponemos el nombre de nuestra base de datos.
});

module.exports = conn; //exportamos conn
```

De esta manera se pondría en index.js:

```
const conn = require("./bd"); //se importa desde afuera del index.
```

Asignamos el puerto.

```
const PORT = 1505; //puede ser cualquiera que no esté en uso.
```

Luego configuramos el servidor para ser iniciado.

```
app.listen(PORT, () => {
    console.log(`Server corriendo en: http://localhost:${PORT}`)
});
```

Express usará el puerto 1505 para nuestro servidor

➔ Rutas en express:

Estructura de una ruta en express

```
app.#tipo(get | post | put | delete('/#nombre de ruta', (req, res) => {
    //contenido de la ruta.
    res.send('Hola desde la ruta'); // para vistas
    //,
    res.json(#objeto); // para api devuelve un json.
    //o
    res.status(#numero); // para devolver un estado del servidor.
});
```

La arrow function recibe req: request y res: response, req es lo que recibe el servidor, como método get (datos en la url) o post (datos enviados por un form), y res es lo que vamos a devolver, res.send() muestra un mensaje en pantalla, res.json(#objeto) devuelve un objeto en formato json, también se pueden usar plantillas como EJS (similar a php con html) con res,render(#nombre) y res.status() devuelve un problema relacionado con el servidor (para más información de los tipos de status visitar: https://http.cat/), tambien puede ir acompañado con json() para mensajes específicos.

Se puede poner las rutas de la api en el mismo archivo o en otro archivo, usaremos otro archivo para usar Router en un archivo llamado productos.js (de la base de datos productos_db) tenemos:

```
const { Router } = require('express'); //metodo de express.
const router = Router(); //creamos router similar a app.
const conn = require('../bd'); // importamos la llamada a la bd.
```

Router nos ayuda a separa las rutas en varios archivos para poder aprovechar la modularidad que ofrece express

Métodos:
Usaremos async en las funciones porque conn.query() devuelve una promise que await recupera.

GET

- Para listar:

```
router.get('/api/productos', async (req, res) => {
    try
    {
        const [rows] = await conn.query(`SELECT * FROM producto`);
        if (rows.length === 0) return res.status(404).json({
error: 'Tabla no encontrada' });
        res.json(rows);
    }
    catch (error)
    {
        res.status(500).json({ error: 'Error al obtener los datos'
});
    }
});
```

Usamos [] en rows para desestructurar y cargar el dato directamente sino nos da datos innecesarios para nuestra api.

- Para seleccionar uno en específico:

```
router.get('/api/productos/:id', async (req, res) => {
    try
    {
        const [rows] = await conn.query('SELECT * FROM producto
WHERE id = ?', [req.params.id]);
        if (rows.length === 0) return res.status(404).json({
error: 'Usuario no encontrado' });
        res.json(rows);
    }
    catch (error)
    {
        res.status(500).json({ error: 'Error al obtener usuario'
});
    }
});
```

Con createPool en conn podemos tener en query la siguiente estructura:
conn.query(#consulta: string, [#valores a reemplazar por ? en #consulta]: array)
podemos tener en la consulta cuantos '?' queramos pero si agregamos al menos uno debemos usar el segundo parámetro de query que es un arreglo para recibir los valores a reemplazar por ? en el mismo orden que se pone en la consulta.
ej. "#consulta valor_1 = ?, valor_2 = ?, valor_3 = ?", [valor2, valor1, valor3]
valor2 tomaría el lugar del primer ?, valor1 del segundo ? y así sucesivamente.

al usar `'/api/productos/:id'` estamos indicando que recibiremos un parámetro de tipo get, por ejemplo al poner la ruta http://localhost:1505/api/shippers/6. estamos indicando que el 6 es un parámetro que obtenemos al usar req.params.id, el nombre el atributo de req.params debe ser el mismo que se pone después del /:#nombre en la ruta, en ese caso sería req.params.#nombre.

POST

```
router.post('/api/productos', async (req, res) => {
    try
    {
        const { nombre, descripcion, precio, stock } = req.body;
        const [rows] = await conn.query('INSERT INTO
producto(nombre, descripcion, precio, stock) VALUES(?,?,?,?)', [
nombre, descripcion, precio, stock ]);
        if (rows.length === 0) return res.status(404).json({
error: 'Usuario no encontrado' });
        res.json(rows);
    }
    catch (error)
    {
        res.status(500).json({ error: 'Error al obtener usuario'
});
    }
});
```

De req.body, que es un tipo de dato que nos da los valores que se envía en el método post y por lo general es un objeto que contiene un json, pero si viene de un post contiene los valores de la etiqueta con el atributo name del mismo nombre
Supongamos tenemos el siguiente form:

```
<form action="/api/productos" method="post">
     <input type="text" name="ProductName" required>
      <input type="text" name="description" required>
      <button type="submit">Guardar</button>
</form>
```

al pasar a /api/productos, req.boy sería igual a:
{
ProductName: input value
description: input value
}
el atributo sería el mismo nombre de los names del form y el valor seria el valor de dicha etiqueta asociada al name respectivo.
Después lo desestructuramos para tener sus valor en 2 constantes para usarlo en la query como valores para los ‘?’.

PUT

```
router.put('/api/productos', async (req, res) => {
    try
    {
        const { id, nombre, descripcion, precio, stock } =
req.body;
        const [rows] = await conn.query('UPDATE producto SET
nombre=?, descripcion=?, precio=?, stock=? WHERE id=?', [ nombre,
descripcion, precio, stock, id ]);
        if (rows.length === 0) return res.status(404).json({
error: 'Usuario no encontrado' });
        res.json(rows);
    }
    catch (error)
    {
        res.status(500).json({ error: 'Error al obtener usuario'
});
    }
});
```

DELETE

```
router.delete('/api/productos/:id', async (req, res) => {
    try
    {
        const [rows] = await conn.query('DELETE FROM producto
WHERE id = ?', [req.params.id]);
        if (rows.length === 0) return res.status(404).json({
error: 'Usuario no encontrado' });
        res.json(rows);
    }
    catch (error)
    {
        res.status(500).json({ error: 'Error al obtener usuario'
});
    }
});
```

Una vez ya creado todo en el archivo productos.js lo meteremos en un carpeta llamada models y para usarlo se tiene que enlazar al index.js.

```
app.use(require(path.join(__dirname, 'models/productos')));
```

__dirname: dirección actual del archivo en la maquina donde este el proyecto.
path.join(#ruta1, #ruta2): método para unir rutas.

Este middleware permite usar productos.js de models en página principal como un modulo que exporta rutas con esta línea de código al después de las rutas en productos.js.

```
module.exports = router; //permite exportar las rutas que creamos.
```

Eso sería lo básico para poder hacer API Rest con Node.js y pnpm.

➔ Servidor web embebido

El comando php -S 0.0.0.0:8080 -t web en composer activa un servidor web embebido que trae php.
0.0.0.0:#puerto: Indica que el servidor escuchará en todas las interfaces de red (0.0.0.0) para las IP que estén en la misma red que nuestro equipo y en el puerto especificado.

Este es para PHP con YII2 en el caso de Node.js Con express se haría lo siguiente:

```
app.listen(PORT, '0.0.0.0', () => {
    console.log(`Server corriendo en: http://0.0.0.0:${PORT}`)
});
```

Sería colocar como parámetro de listen (función que ayuda a asignar un puerto a nuestro proyecto de express) ‘0.0.0.0’ para que todas las máquinas que estén nuestra misma red puedan acceder a la url de nuestro proyecto para acceder a las APIs, en nuestro caso : “http://192.168.#numero.#número:PORT/api/productos”.